

Travel Application

Front-End Developer: Assignment

Build a travel web application that helps someone explore destinations around the world, see what the weather is like there right now, discover the famous places worth visiting, and plan a trip with the help of an AI assistant.

Which destinations you include, how you structure your data, which APIs you choose and how the whole thing looks are **your decisions**. Working that out is part of the assignment.

WHAT TO BUILD

01 A landing experience

Open with a hero that uses a looping background video. It should make someone want to keep scrolling.

02 Destination explorer

A visitor can browse destinations, search them, and filter them. Opening one gives it a page of its own with everything worth knowing about the place.

03 Famous places

Each destination shows the notable places to visit there, presented properly: not as a bare list of names.

04 Location awareness

Ask the visitor for their location and use it. Also let them choose a location by searching for it, so the app is useful whether or not they share it.

05 Real-time weather

Integrate an API that shows live weather for a location. You can use OpenWeather.

06 Images

Destination and place images should be fetched from an image source, not hardcoded into your project. You can use Pexels or Unsplash.

07 An AI chatbot

Integrate a conversational assistant a visitor can ask questions about a destination: how long to spend there, what to see, when to go. You can use the Google Gemini API.

08 Itinerary planning

The assistant should be able to plan a trip. Give the visitor a clear way to generate an itinerary, and **render the result on the page as a real, readable day-by-day plan**: not as a block of chat text.

WHAT WILL BE LOOKED AT

- **Visual design.** Layout, typography, spacing, colour, alignment, restraint. This carries the most weight: we are a design-led company.
- **Motion and interaction.** Transitions, hover and focus states, how the page arrives. Used with intent, not sprinkled on.
- **Responsiveness.** It should hold together from a phone to a large screen.
- **What happens when things go wrong.** Loading, empty results, a denied location permission, a failed request. Every one of these is a state you design, not an accident.
- **Accessibility.** Semantic markup, readable contrast, and the whole thing usable by keyboard.
- **Code quality.** Sensible structure, clear naming, nothing dead left behind.

RULES

- Build it in **React**.
 - The interface must be **clean and well presented**. Considered layout, spacing and typography: not a default-looking page.
 - It must be **fully responsive** and work properly on a phone, a tablet and a desktop.
 - **Never commit API keys to your repository.** Use environment variables and keep them out of version control.
 - The work must be your own. Two submissions that are substantially the same are both rejected.
-

WHAT TO SUBMIT

- **A live deployed link.** Your application must be deployed and working. Open it yourself in a private window before you send it.
- **A public GitHub repository.** Public, not private.
- **A README** in that repository containing a description of the project, screenshots of it running, the features you built, the APIs you used, and how to run it locally.

Once your project is built and deployed, **fill the form attached in your email** to share your live link.

RESOURCES YOU MAY FIND USEFUL

Weather: OpenWeather · **AI:** Google Gemini · **Images:** Pexels, Unsplash · **Video:** Coverr, Mixkit · **Deployment:** Vercel, Netlify, GitHub Pages

You are not limited to these. If you find something better, use it and say so in your README.